1. Banker's Algorithm

```c
#include <stdio.h>

int main() {
    int n, m, i, j;

    printf("Enter number of processes: \n");
    scanf("%d", &n);

    printf("Enter number of resource types: \n");
    scanf("%d", &m);

    int alloc[n][m], max[n][m], need[n][m];
    int avail[m], finish[n], safeSeq[n];

    printf("Enter Allocation Matrix:\n");
    for(i = 0; i < n; i++) {
        for(j = 0; j < m; j++) {
            scanf("%d", &alloc[i][j]);
        }
    }

    printf("Enter Max Matrix:\n");
    for(i = 0; i < n; i++) {
        for(j = 0; j < m; j++) {
            scanf("%d", &max[i][j]);
        }
    }
```

```c
    printf("Enter Available Resources: \n");
    for(j = 0; j < m; j++) {
        scanf("%d", &avail[j]);
    }

    for(i = 0; i < n; i++) {
        for(j = 0; j < m; j++) {
            need[i][j] = max[i][j] - alloc[i][j];
        }
    }

    for(i = 0; i < n; i++) {
        finish[i] = 0;
    }

    int count = 0;

    while(count < n) {
        int found = 0;

        for(i = 0; i < n; i++) {
            if(finish[i] == 0) {

                int canExecute = 1;

                for(j = 0; j < m; j++) {
                    if(need[i][j] > avail[j]) {
                        canExecute = 0;
```

```
                if(canExecute) {
                    for(j = 0; j < m; j++) {
                        avail[j] += alloc[i][j];
                    }

                    safeSeq[count++] = i;
                    finish[i] = 1;
                    found = 1;
                }
            }
        }

        if(found == 0) {
            break;
        }
    }

    if(count == n) {
        printf("System is in a SAFE STATE.\n");
        printf("Safe Sequence: ");
        for(i = 0; i < n; i++) {
            printf("P%d ", safeSeq[i]);
        }
        printf("\n");
    } else {
        printf("System is NOT in a safe state (Deadlock may occur).\n");
    }
```

Custom Test — Success

Input :

```
3
3
0 1 0
2 0 0
3 0 3
3 2 2
2 1 1
4 3 3
0 0 0
```

Expected Output :

```
Enter number of processes:
Enter number of resource types:
Enter Allocation Matrix:
Enter Max Matrix:
Enter Available Resources:
System is NOT in a safe state (Deadlock may occur).
```

Output :

```
Enter number of processes:
Enter number of resource types:
Enter Allocation Matrix:
Enter Max Matrix:
Enter Available Resources:
System is NOT in a safe state (Deadlock may occur).
```